Transaction Database Cluster

Single Design Document

Aurora PostgreSQL Global Database Architecture

Document ID:	SDD-TDB-2026-004
Version:	1.2
Date:	May 5, 2026
Classification:	Internal - Confidential
Owner:	Database Platform Engineering
Status:	Approved

Version History

Version	Date	Author(s)	Description of Changes
1.2	05/05/2026	J. Park, L. Fernandez	Added CDC pipeline details, updated capacity numbers
1.1	02/20/2026	L. Fernandez	Global secondary region deployment documentation
1.0	11/15/2025	J. Park, A. Volkov	Initial document creation

Table of Contents

1 Purpose of this Document
- 1.1 Scope
- 1.2 Intended Audience

2 High Level Architecture
- 2.1 Aurora Global Database Overview
- 2.2 Architecture Principles
- 2.3 System Context

3 Cluster Topology
- 3.1 Writer Instance
- 3.2 Reader Instances
- 3.3 Global Secondary Cluster
- 3.4 On-Premises Logical Replica

4 Aurora Storage Architecture
- 4.1 Six-Copy Replication
- 4.2 Quorum Protocol
- 4.3 Self-Healing Storage

5 High Availability & Replication
- 5.1 Intra-Region Failover
- 5.2 Cross-Region Failover
- 5.3 Logical Replication

6 Connection Management
- 6.1 PgBouncer Architecture
- 6.2 ProxySQL Read/Write Splitting
- 6.3 IAM Database Authentication

7 Data Partitioning Strategy
- 7.1 Monthly Range Partitioning
- 7.2 Hash Partitioning (Balances)
- 7.3 Partition Maintenance

8 Data Lifecycle Management

9 Network Architecture
- 9.1 VPC Configuration
- 9.2 Security Groups
- 9.3 Cross-VPC Peering

10 Data Flow
- 10.1 Write Path
- 10.2 Read Path
- 10.3 CDC Pipeline

11 Backup & Point-in-Time Recovery

12 Security & Access Control

13 Performance Tuning

14 Monitoring & Alerting

15 Operational Procedures

16 Known Risks & Deviations

References

1 Purpose of this Document

This Single Design Document provides a comprehensive architectural reference for the Transaction Database Cluster (TDB) operated by Example Organization. The TDB is an Aurora PostgreSQL Global Database deployment that serves as the authoritative system of record for all financial transaction data across the bank's retail, commercial, and wealth management divisions. This document captures the infrastructure design, storage architecture, replication topology, connection management, data lifecycle policies, and operational procedures that govern the platform's operation at scale.

The Transaction Database Cluster is classified as a Tier-0 Business Critical system because it underpins every financial transaction processed by the bank. When the TDB is unavailable, the bank cannot process payments, record account movements, settle interbank transfers, or generate regulatory reports. The system currently manages 18 billion transaction records spanning 340 terabytes of storage, processing a sustained peak of 28,000 write operations per second and 95,000 read operations per second during business hours. The annual growth rate of 42% in both volume and throughput demands a design that scales horizontally without requiring application-level changes.

The decision to deploy Aurora PostgreSQL rather than a traditional self-managed PostgreSQL cluster was driven by three primary factors. First, Aurora's storage architecture provides six-copy replication across three Availability Zones with quorum-based writes that eliminate the risk of data loss during infrastructure failures -- a non-negotiable requirement for financial transaction data subject to OCC Heightened Standards and Regulation E. Second, Aurora's Global Database capability provides sub-second cross-region replication without the operational complexity of managing logical replication slots, WAL shipping, and failover orchestration manually. Third, Aurora's separation of compute and storage allows independent scaling of read capacity (by adding reader instances) and storage capacity (which auto-scales transparently), enabling the platform to grow without the disruptive resize operations required by traditional database deployments.

1.1 Scope

This document covers the end-to-end architecture of the Transaction Database Cluster including the Aurora PostgreSQL Global Database deployment across us-east-1 (primary) and us-west-2 (disaster recovery), the connection management layer (PgBouncer and ProxySQL), the change data capture pipeline (Debezium to Kafka), the on-premises logical replica for legacy system access, data partitioning and lifecycle management, network architecture, security controls, performance tuning methodology, monitoring and alerting configuration, and operational runbook procedures.

The scope explicitly excludes the application services that connect to the database, which are documented in their respective SDDs (SDD-001 Core Banking Platform, SDD-002 Payment Gateway Service). It also excludes the downstream consumers of the CDC stream, though their integration patterns and latency requirements are referenced where relevant to architectural decisions affecting the CDC pipeline configuration.

1.2 Intended Audience

This document is intended for database reliability engineers, cloud infrastructure architects, application developers who interact with the transaction database, security and compliance teams responsible for data protection controls, and operations staff responsible for maintaining the platform. It serves as the authoritative reference during capacity planning exercises, failover drills, performance investigations, compliance audits, and incident response activities involving transaction data.

2 High Level Architecture

2.1 Aurora Global Database Overview

The Transaction Database Cluster is built on Amazon Aurora PostgreSQL Global Database, a purpose-built relational database engine that combines the performance and availability of high-end commercial databases with the simplicity and cost-effectiveness of open-source PostgreSQL. Aurora's architecture fundamentally separates the compute layer (database instances running the PostgreSQL query engine) from the storage layer (a distributed, fault-tolerant storage subsystem that replicates data six ways across three Availability Zones). This separation enables several capabilities critical to the transaction workload.

The Global Database configuration extends the primary cluster in us-east-1 with a secondary cluster in us-west-2 that maintains a full copy of the storage volume with sub-second replication lag. This secondary cluster provides two key capabilities: it serves as a disaster recovery target that can be promoted to primary within 60 seconds (RPO < 1 second, RTO < 60 seconds), and it enables low-latency reads for applications deployed in the western United States that would otherwise incur 60-80ms of cross-region latency accessing the primary cluster. The replication between regions operates at the storage layer, bypassing the database engine entirely, which means it adds zero overhead to the primary cluster's write performance.

The architecture also includes an on-premises PostgreSQL 15 instance at DC-East that receives a subset of transaction data via logical replication. This replica exists solely to support legacy mainframe applications that cannot be modified to connect to cloud-hosted databases within the current modernization timeline. The logical replica receives real-time updates via AWS Direct Connect and serves approximately 3,400 queries per second from five legacy applications scheduled for decommissioning between Q3 2026 and Q2 2027.

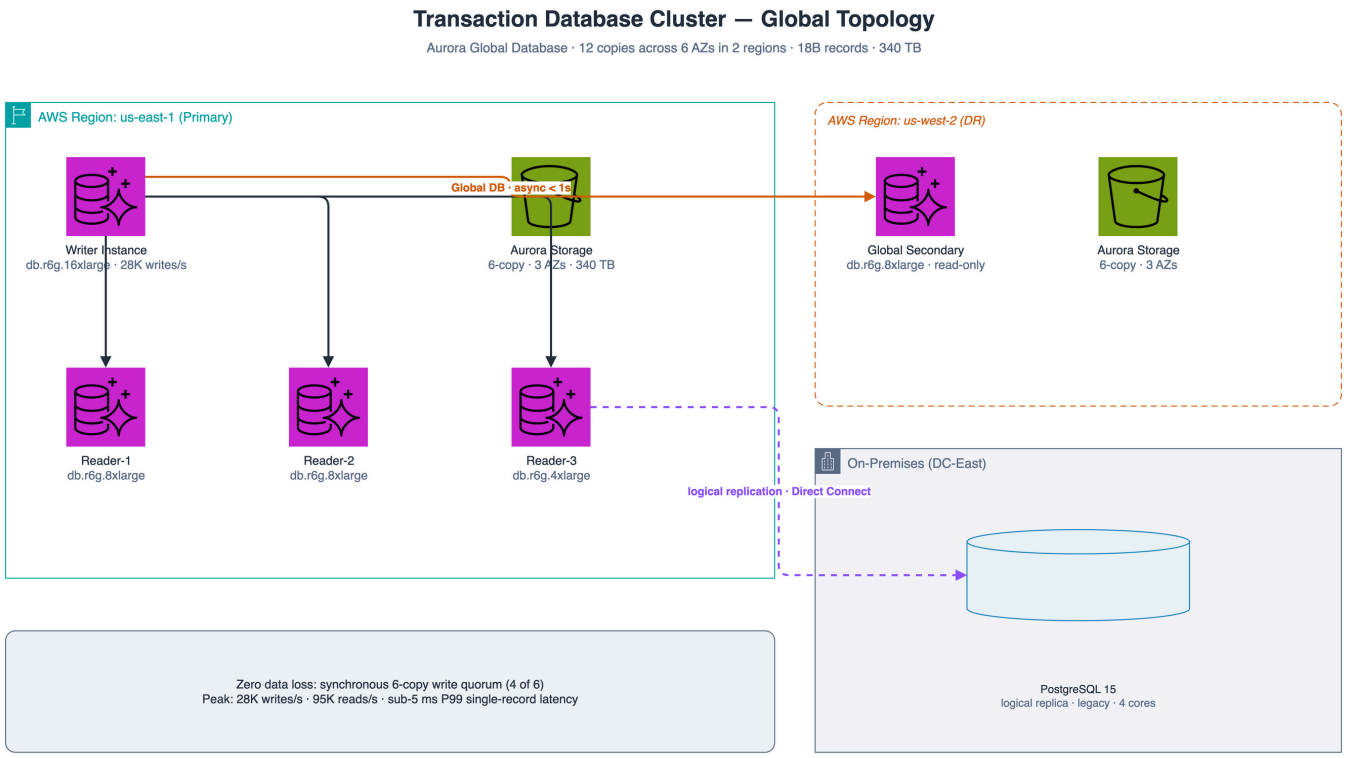


Figure 1: Transaction Database Cluster - Global Topology

2.2 Architecture Principles

The Transaction Database Cluster architecture adheres to the following principles that guide all design decisions and trade-off resolutions:

Zero Data Loss for Financial Transactions: Every committed transaction must be durably stored with zero risk of loss, even during infrastructure failures. This principle manifests in Aurora's synchronous six-copy write quorum (4 of 6 copies must acknowledge before commit returns to the client), continuous backup to S3 with 5-minute point-in-time recovery granularity, and the Global Database configuration that maintains an additional six copies in the DR region. The total redundancy factor is 12 copies across 6 Availability Zones in 2 regions.

Predictable Low Latency Under Load: The database must deliver consistent sub-5ms P99 latency for single-record operations regardless of concurrent load. This is achieved through connection pooling (PgBouncer limiting active connections to match CPU cores), read/write splitting (ProxySQL routing reads to dedicated reader instances), and careful workload isolation (analytics queries routed to a dedicated reader that does not serve OLTP traffic).

Horizontal Read Scalability: Read capacity must scale linearly by adding reader instances without application changes. Aurora's reader endpoint provides automatic load balancing across all readers, and ProxySQL enables application-transparent routing. When a new reader is added, it begins serving traffic within minutes without any data copying (it shares the same distributed storage volume).

Operational Simplicity: The architecture minimizes the operational burden on the database reliability engineering team by leveraging managed services wherever possible. Aurora handles storage management, replication, backup, patching (minor versions), and failure detection. This allows the team to focus on schema optimization, query performance, and capacity planning rather than infrastructure maintenance.

2.3 System Context

The Transaction Database Cluster sits at the heart of the organization's transaction processing ecosystem, receiving writes from 8 upstream application services and serving reads to 23 downstream consumers via the CDC pipeline. The primary write sources include the Core Banking Platform (account transactions), Payment Gateway Service (card and ACH payments), Wire Transfer Service (domestic and international wires), Direct Deposit Service (payroll and government payments), Loan Servicing Platform (loan payments and disbursements), Treasury Management System (corporate cash management), Internal Transfer Engine (between-account movements), and Fee Assessment Service (service charges and interest).

Downstream CDC consumers include the fraud detection engine (real-time transaction scoring), balance aggregation service (maintaining current balances from transaction journal), statement generation (monthly customer statements), regulatory reporting (BSA/AML, CTR, SAR filing), data warehouse (historical analytics), customer notification service (transaction alerts), and 17 additional internal consumers ranging from marketing analytics to operational dashboards. Each consumer connects to the Kafka CDC stream rather than querying the database directly, which isolates the database from the read amplification that would otherwise result from 23 separate polling applications.

3 Cluster Topology

3.1 Writer Instance

The primary writer instance is a db.r6g.16xlarge providing 64 vCPUs (AWS Graviton2 ARM-based processors) and 512 GB of RAM. The Graviton2 processor was selected over Intel-based alternatives for its 40% better price-performance ratio on database workloads, validated through extensive benchmarking against the bank's actual transaction mix. The 512 GB buffer pool provides a 98.7% cache hit ratio for the hot working set, meaning that only 1.3% of read operations require storage I/O.

The writer instance handles all INSERT, UPDATE, and DELETE operations for the transaction database, processing a sustained peak of 28,000 write operations per second during the 11:00 AM - 2:00 PM ET business window when payment processing, ACH settlement, and wire transfer volumes converge. Off-peak write volume drops to approximately 4,200 operations per second (overnight hours) with a 24-hour average of 12,400 writes per second.

The writer instance also serves a limited number of read operations that require strong consistency (read-after-write guarantee) -- specifically, balance checks that immediately follow a debit transaction and fraud scoring queries that must see the most recent transaction. These strongly-consistent reads account for approximately 8% of total read volume and add approximately 2,200 reads per second to the writer's workload. All other reads are routed to reader instances via ProxySQL, which provide eventual consistency with a typical replication lag of less than 20 milliseconds.

The writer instance is configured with PostgreSQL parameters optimized for write-heavy OLTP workloads: shared_buffers at 128 GB (25% of RAM), effective_cache_size at 384 GB, work_mem at 256 MB, maintenance_work_mem at 2 GB, wal_buffers at 64 MB, and max_connections at 5000 (though actual connection count is managed by PgBouncer at a much lower level). The checkpoint_timeout is set to 15 minutes with checkpoint_completion_target of 0.9 to spread checkpoint I/O over the full interval.

3.2 Reader Instances

The primary cluster includes three reader instances that handle the bulk of read traffic, collectively processing up to 95,000 read operations per second at peak. The readers are intentionally heterogeneous to serve different workload profiles:

Reader-1 (db.r6g.8xlarge, 32 vCPU, 256 GB): Dedicated to the Core Banking Platform's real-time queries -- account lookup, transaction history retrieval, and balance inquiries. This reader handles approximately 45,000 reads per second with a P99 latency requirement of 8ms. The instance is sized to maintain a 97% buffer pool hit ratio for the most-queried account data.

Reader-2 (db.r6g.8xlarge, 32 vCPU, 256 GB): Serves the Payment Gateway, Wire Transfer, and other transactional services that require sub-10ms reads for payment validation and duplicate detection. This reader handles approximately 35,000 reads per second with a P99 latency requirement of 10ms.

Reader-3 (db.r6g.4xlarge, 16 vCPU, 128 GB): Dedicated analytics reader that serves long-running queries from the reporting service, data warehouse extract jobs, and ad-hoc analytical queries from the finance team. This reader is intentionally isolated from OLTP traffic to prevent expensive analytical scans from consuming buffer pool capacity needed by latency-sensitive queries. Queries on this reader may run for minutes without impacting OLTP performance.

All readers share the same underlying Aurora storage volume as the writer, meaning they see writes with a typical replication lag of 10-20 milliseconds (Aurora's in-region replication operates at the storage layer with sub-millisecond physical replication, but the reader's buffer cache requires a brief invalidation cycle). The reader endpoint provides DNS-based load balancing across Reader-1 and Reader-2, while Reader-3 is accessed via a custom endpoint that routes only analytics traffic.

3.3 Global Secondary Cluster

The disaster recovery cluster in us-west-2 consists of a single db.r6g.8xlarge instance that receives data from the primary cluster via Aurora's Global Database replication mechanism. This replication operates at the storage layer, shipping redo log records from the primary's storage volume to the secondary's storage volume with a typical lag of 400-800 milliseconds (measured as the time between a write committing in us-east-1 and becoming readable in us-west-2).

Under normal operations, the secondary cluster serves read-only traffic from applications deployed in the western US region, providing local-latency access (2-5ms) rather than the 60-80ms cross-region latency that would be incurred by routing to the primary. Approximately 8,000 reads per second are served from the secondary cluster, primarily from the West Coast mobile banking infrastructure and the San Francisco trading desk applications.

In a disaster recovery scenario, the secondary cluster can be promoted to a standalone primary cluster within 60 seconds. The promotion process involves: (1) detaching the secondary from Global Database replication (< 5 seconds), (2) promoting the reader instance to a writer (< 30 seconds), and (3) updating DNS records to point the cluster endpoint to the new primary (< 25 seconds via Route 53 health-check-based failover). After promotion, additional reader instances are automatically launched by an Auto Scaling policy to match the read capacity of the original primary cluster.

3.4 On-Premises Logical Replica

An on-premises PostgreSQL 15 instance at DC-East receives a filtered subset of transaction data via native PostgreSQL logical replication over AWS Direct Connect. This replica exists to support five legacy mainframe-adjacent applications that cannot be modified to connect to cloud-hosted databases: the legacy general ledger posting system, the mainframe reconciliation batch, the regulatory capital calculation engine, the branch teller overnight batch, and the legacy statement rendering service.

The logical replication publication is configured to replicate only the core transactions table and the account_balances table -- a subset representing approximately 15% of the total database volume. This selective replication reduces the bandwidth requirement to approximately 180 Mbps sustained over the Direct Connect link and limits the on-premises storage requirement to 52 TB. The replica is configured with 4 physical CPU cores (Intel Xeon), 64 GB RAM, and NVMe SSD storage, sufficient for the 3,400 queries per second workload from the legacy applications.

The logical replication connection uses a dedicated replication slot on the Aurora writer instance with a WAL retention policy of 24 hours. If the on-premises replica becomes unavailable for more than 24 hours (e.g., due to a prolonged Direct Connect outage), the replication slot is automatically dropped to prevent WAL accumulation on the writer, and a full re-synchronization is required upon reconnection. This scenario has occurred twice in the past 12 months (both during planned Direct Connect maintenance windows) and the re-synchronization completed within 4 hours each time.

4 Aurora Storage Architecture

Aurora's storage layer is a purpose-built distributed storage service that provides the durability, availability, and performance guarantees required for financial transaction data. Unlike traditional PostgreSQL where the database engine manages storage directly on attached EBS volumes, Aurora delegates all storage operations to a separate distributed subsystem that operates independently of the database compute instances. This section describes the storage architecture in detail because understanding it is essential for capacity planning, failure analysis, and performance optimization.

4.1 Six-Copy Replication

Every write to the Aurora storage volume is synchronously replicated to six copies distributed across three Availability Zones (two copies per AZ). The storage volume is divided into 10 GB segments called Protection Groups, and each Protection Group independently maintains its six copies. With 340 TB of logical data, the cluster contains approximately 34,000 Protection Groups, each independently fault-tolerant.

The six-copy design provides three critical properties. First, it tolerates the loss of an entire Availability Zone (losing 2 of 6 copies) plus one additional node failure in a surviving AZ (total 3 of 6 copies lost) without any data loss -- exceeding the durability requirements of even the most stringent financial regulations. Second, it enables reads to be served from the nearest available copy, reducing read latency to sub-millisecond for cached data. Third, it provides the redundancy needed for background repair operations to complete without impacting availability.

The effective durability of the six-copy architecture is 99.999999999% (11 nines) annualized, meaning the expected time to data loss is approximately 100 billion years. This calculation accounts for the mean time between disk failures, the repair time for a 10 GB Protection Group (approximately 10 seconds using peer-to-peer repair), and the probability of correlated failures across multiple AZs. For context, S3's published durability (also 11 nines) uses a similar multi-AZ replication model.

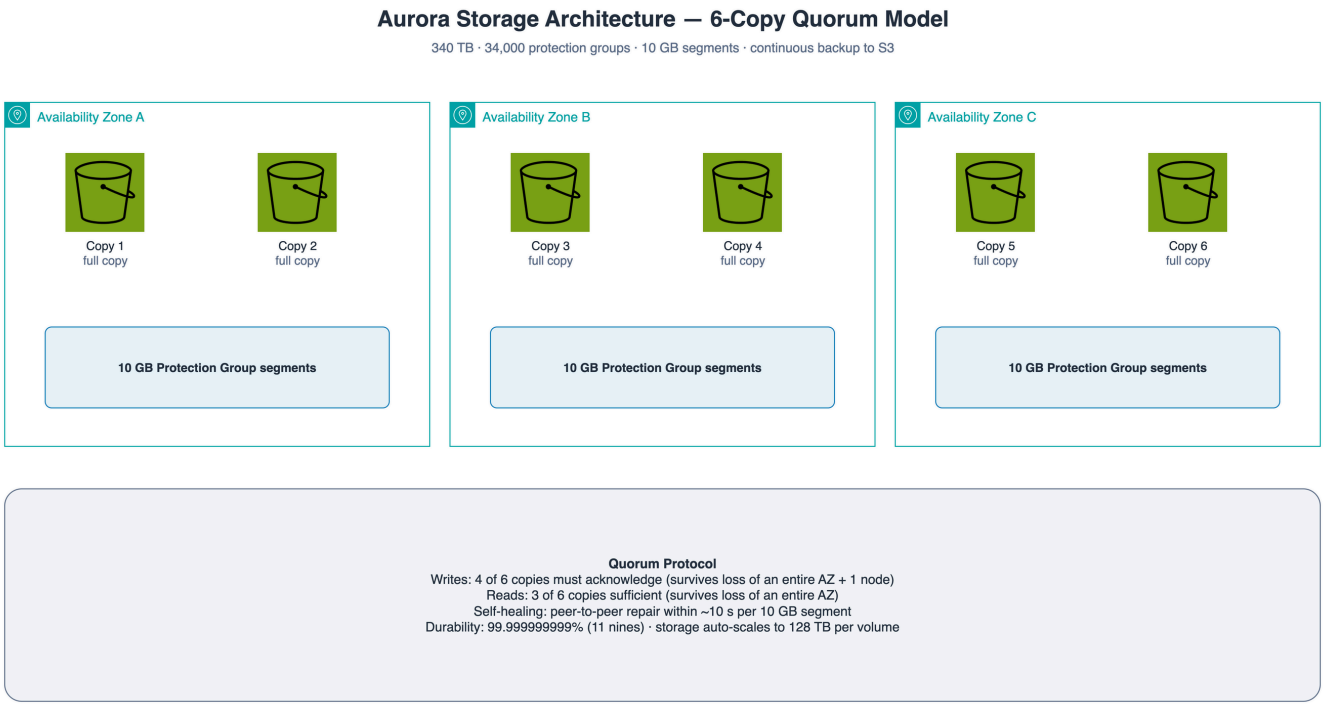


Figure 2: Aurora Storage Architecture - 6-Copy Quorum Model

4.2 Quorum Protocol

Aurora uses a quorum-based protocol for both reads and writes that provides consistency without the performance penalty of requiring all copies to acknowledge every operation. The write quorum requires 4 of 6 copies to acknowledge before the write is considered durable and the commit can be returned to the client. The read quorum requires 3 of 6 copies to agree on the most recent version of a page.

The 4-of-6 write quorum means that Aurora can continue accepting writes even if an entire Availability Zone is lost (2 copies unavailable), because 4 copies remain in the surviving 2 AZs. It can also tolerate one additional node failure beyond the AZ loss before writes are blocked. The 3-of-6 read quorum means that reads remain available even during write quorum degradation, as only 3 responding copies are needed.

The quorum protocol operates at the Protection Group level (10 GB segments), not at the volume level. This means that a disk failure affecting one Protection Group does not impact the quorum status of other Protection Groups on different disks. The blast radius of any single disk failure is limited to the Protection Groups stored on that disk, and repair begins immediately via peer-to-peer copying from healthy copies. During the repair window (approximately 10 seconds for a 10 GB segment), the affected Protection Group operates with reduced redundancy (5 of 6 copies) but remains fully available for both reads and writes.

4.3 Self-Healing Storage

Aurora's storage layer continuously monitors the health of all data copies and automatically initiates repair operations when a copy becomes unavailable or corrupted. The repair mechanism uses peer-to-peer copying between storage nodes within the same Protection Group, meaning that repair traffic does not traverse the database compute instances or impact query performance.

When a storage node fails, the impacted Protection Groups (typically hundreds to thousands, depending on the node's capacity) all begin repair simultaneously. Each 10 GB segment is repaired independently by copying from a surviving peer copy to a new location on a healthy storage node. The repair throughput is approximately 1 GB per second per Protection Group, meaning a 10 GB segment is fully repaired in approximately 10 seconds. Even a large storage node failure affecting 5 TB of data across 500 Protection Groups completes repair within approximately 10 seconds because all segments repair in parallel.

The self-healing mechanism also detects and repairs silent data corruption using checksums computed at write time and verified at read time. If a read returns data whose checksum does not match the expected value, Aurora automatically reads from an alternative copy and initiates a background repair of the corrupted copy. Over the past 12 months, the storage layer has detected and automatically repaired 7 instances of silent corruption -- none of which were visible to the application or required operator intervention.

5 High Availability & Replication

5.1 Intra-Region Failover

Within the us-east-1 primary region, Aurora provides automatic failover from the writer instance to one of the reader instances if the writer becomes unavailable. The failover process is managed entirely by Aurora's cluster management layer and completes within 30-60 seconds, during which write operations are unavailable but read operations continue to be served by the remaining readers.

The failover priority is configured to prefer Reader-1 (db.r6g.8xlarge) as the first failover target because it matches the writer's memory configuration most closely, minimizing the buffer pool warm-up time after promotion. Reader-2 is the second priority, and Reader-3 is excluded from failover candidacy because its smaller instance size (db.r6g.4xlarge) cannot sustain the full write workload. After failover, a replacement reader instance is automatically launched to restore the original cluster topology.

The failover mechanism detects writer unavailability through a heartbeat mechanism with a timeout of 10 seconds. If the writer fails to respond to three consecutive heartbeats (30 seconds total), the cluster initiates failover. The actual promotion of the reader to writer takes approximately 15-30 seconds, during which the new writer applies any pending redo log entries to ensure it has the most recent committed data before accepting new writes. The total observed failover time in quarterly testing ranges from 35 to 52 seconds.

Applications connected via PgBouncer experience the failover as a brief connection error followed by automatic reconnection. PgBouncer's server_login_retry parameter is configured to 1 second, meaning it will attempt to re-establish server connections every second until the new writer is available. Applications using connection pooling at the application level (such as HikariCP in Java services) are configured with a connection timeout of 60 seconds to accommodate the full failover window without returning errors to end users.

5.2 Cross-Region Failover

Cross-region failover from us-east-1 to us-west-2 is invoked when the entire primary region becomes unavailable or when a regional AWS outage is declared. Unlike intra-region failover (which is automatic), cross-region failover is a manual decision requiring approval from the on-call Database Reliability Engineer and the incident commander. This manual gate exists because cross-region failover involves a brief data loss window (the replication lag, typically 400-800ms of committed transactions) and because premature failover can complicate recovery when the primary region returns.

The cross-region failover procedure consists of four steps: (1) Confirm that the primary region is genuinely unavailable and not experiencing a transient issue -- the runbook requires 5 minutes of confirmed unavailability before proceeding. (2) Initiate Global Database detach, which breaks the replication link between regions and promotes the secondary to an independent cluster. (3) Promote the secondary reader to a writer role, enabling write operations in us-west-2. (4) Update Route 53 DNS records to point the global cluster endpoint to the new primary in us-west-2.

After the primary region recovers, the procedure to re-establish Global Database replication requires creating a new secondary cluster in us-east-1 from a snapshot of the (now-primary) us-west-2 cluster. This process takes approximately 4-6 hours for the 340 TB storage volume and runs in the background without impacting the new primary's performance. Once the new secondary is synchronized, the topology can optionally be reversed back to the original configuration

(us-east-1 primary) during a maintenance window, or the us-west-2 cluster can remain as the permanent primary if the original primary cannot be trusted.

5.3 Logical Replication

The on-premises PostgreSQL replica uses native PostgreSQL logical replication (pgoutput plugin) rather than Aurora's physical replication because the target is a standard PostgreSQL instance, not an Aurora instance. Logical replication operates at the SQL level -- capturing individual INSERT, UPDATE, and DELETE operations and replaying them on the subscriber -- rather than at the storage level like Aurora's internal replication.

The logical replication publication on the Aurora writer includes two tables: core_transactions (the primary transaction journal, 16.8 billion rows, monthly range partitioned) and account_balances (current balance state, 94 million rows, hash partitioned). The publication uses the FOR ALL TABLES IN SCHEMA syntax scoped to the public schema, with row filters that exclude transaction records older than 24 months to limit the replica's storage footprint. This filtering reduces the replicated volume from the full 18 billion rows to approximately 4.2 billion active rows.

The replication lag to the on-premises replica is typically 2-8 seconds under normal conditions, primarily driven by the network latency of the Direct Connect link (2.1ms one-way) and the overhead of logical decoding on the writer. During peak write periods (28,000 writes/second), the lag may increase to 15-20 seconds as the logical decoding plugin processes the higher WAL volume. This lag is acceptable for the legacy applications' requirements, which specify a maximum staleness tolerance of 60 seconds.

6 Connection Management

Connection management is a critical architectural concern for the Transaction Database Cluster because PostgreSQL's process-per-connection model means that each active database connection consumes approximately 10 MB of RAM and a kernel thread on the database instance. With 8 upstream application services running hundreds of instances each, the naive approach of direct database connections would result in tens of thousands of connections -- far exceeding what the database can efficiently handle. The connection management layer (PgBouncer + ProxySQL) solves this by multiplexing thousands of client connections onto a small pool of server connections.

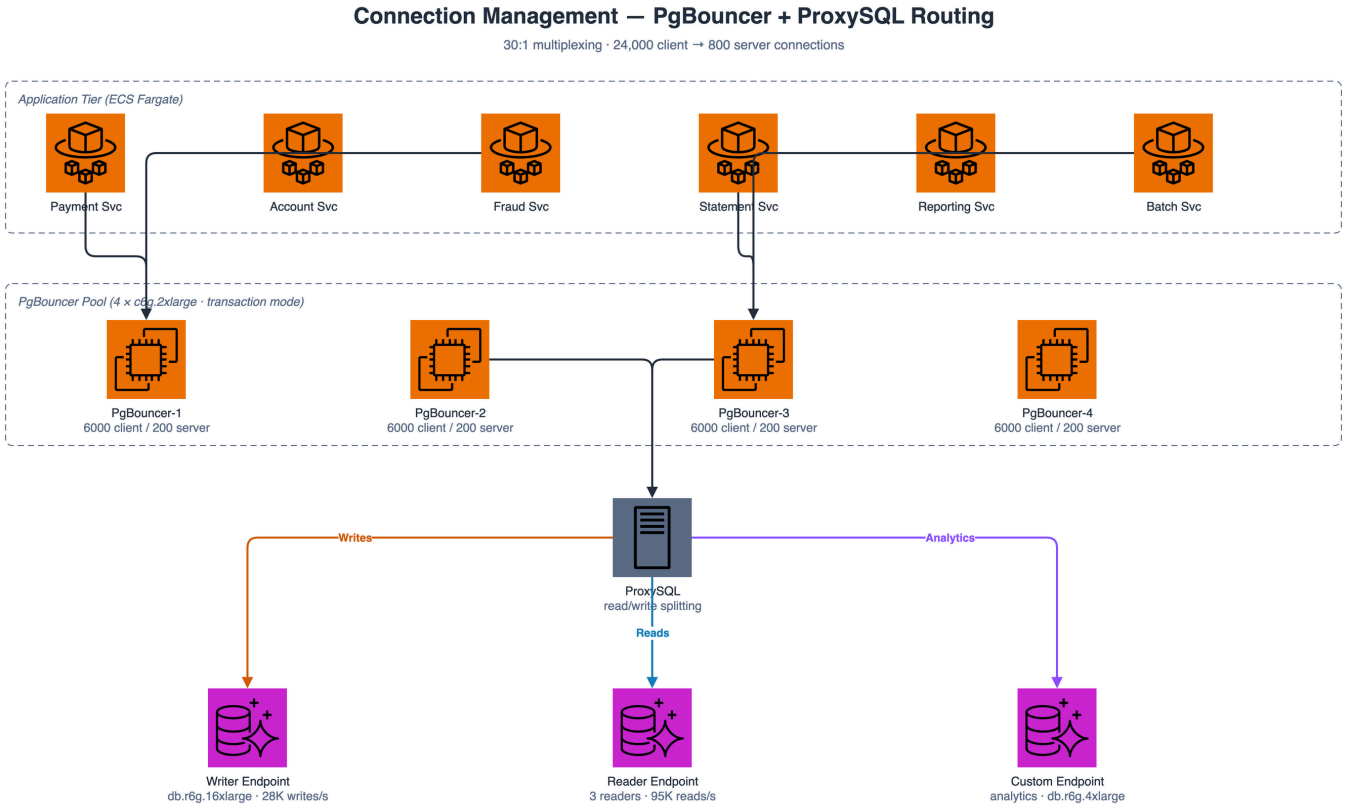


Figure 3: Connection Management - PgBouncer + ProxySQL Routing

6.1 PgBouncer Architecture

PgBouncer is deployed as a fleet of 4 instances running on dedicated c6g.2xlarge EC2 instances (8 vCPU, 16 GB RAM) behind a Network Load Balancer. Each PgBouncer instance is configured to accept up to 6,000 client connections and maintain a pool of 200 server connections to the Aurora writer and reader endpoints. The aggregate capacity across 4 instances is 24,000 client connections multiplexed onto 800 server connections -- a 30:1 multiplexing ratio that dramatically reduces the load on the database instances.

PgBouncer operates in transaction mode, which means a server connection is allocated to a client only for the duration of a transaction (from BEGIN to COMMIT/ROLLBACK) and returned to the pool immediately after. This mode provides the highest multiplexing efficiency because server connections are idle between transactions rather than being held for the entire client session. The trade-off is that session-level features (prepared statements, advisory locks, SET commands) do not persist between transactions. Application services have been designed to be stateless between

transactions to accommodate this constraint.

Each PgBouncer instance is configured with the following pool parameters: default_pool_size=200 (server connections per user/database pair), reserve_pool_size=20 (additional connections for burst handling), max_client_conn=6000, server_idle_timeout=300 (seconds before idle server connection is closed), client_idle_timeout=3600 (seconds before idle client is disconnected), and query_wait_timeout=30 (seconds a client query will wait for a server connection before returning an error). The server_login_retry is set to 1 second to enable rapid reconnection during Aurora failover events.

Health monitoring for the PgBouncer fleet is provided by the Network Load Balancer's health checks (TCP probe on port 6432 every 10 seconds) and by a custom CloudWatch metric that tracks the ratio of active server connections to total pool capacity. If a PgBouncer instance becomes unhealthy, the NLB removes it from the target group within 20 seconds and redistributes its client connections across the remaining instances. Auto Scaling is configured to launch a replacement instance if the fleet drops below 3 healthy instances.

6.2 ProxySQL Read/Write Splitting

ProxySQL sits between PgBouncer and the Aurora endpoints, providing intelligent query routing based on whether a query is a read or a write. All queries beginning with INSERT, UPDATE, DELETE, or enclosed in an explicit transaction (BEGIN...COMMIT) are routed to the Aurora cluster (writer) endpoint. All SELECT queries outside an explicit transaction are routed to the Aurora reader endpoint for load-balanced distribution across the three reader instances.

ProxySQL is deployed as a pair of instances (active-passive with keepalived) on c6g.xlarge EC2 instances. The active instance maintains routing rules defined in its mysql_query_rules table (adapted for PostgreSQL via the PostgreSQL-compatible mode). Query rules use regex pattern matching to classify traffic: write patterns are matched with high priority, and all remaining traffic defaults to the reader hostgroup. Additionally, a custom endpoint for Reader-3 (analytics) is configured as a separate hostgroup that receives traffic only from the reporting service's dedicated credentials.

The read/write split ratio under normal operations is approximately 77% reads to 23% writes. This routing eliminates approximately 73,000 reads per second from the writer instance that would otherwise compete with write operations for CPU and buffer pool resources. Monitoring confirms that the write instance CPU utilization decreased from 82% to 41% after ProxySQL was deployed, with no change in write throughput -- the freed capacity was previously consumed by read queries that are now served by readers.

6.3 IAM Database Authentication

The Transaction Database Cluster uses IAM Database Authentication for all application connections, eliminating the need for long-lived database passwords stored in application configuration. Each application service has a dedicated IAM role that grants the rds-db:connect permission scoped to the specific database user and cluster resource. Authentication tokens are generated by calling the AWS RDS API and are valid for 15 minutes, after which a new token must be generated for new connections.

The IAM authentication flow integrates with PgBouncer via the auth_query mechanism: PgBouncer validates client credentials by executing a query against a custom authentication function that verifies IAM tokens. This allows applications to present IAM-generated tokens to PgBouncer, which then authenticates the client without needing to maintain a separate credential store. The PgBouncer-to-Aurora server connections themselves use IAM authentication with a service role dedicated to the connection pooling layer.

Database-level access control is enforced through PostgreSQL roles that correspond to application service identities. Each service has a dedicated database user with GRANT permissions limited to the specific tables and operations it requires. For example, the Payment Gateway service user has INSERT permission on the core_transactions table and SELECT permission on the account_balances table, but no DROP, ALTER, or TRUNCATE permissions. This least-privilege model ensures that even a compromised application service cannot perform destructive operations beyond its intended scope.

7 Data Partitioning Strategy

Data partitioning is essential for the Transaction Database Cluster because a single unpartitioned table containing 18 billion rows would be unmanageable for both query performance and operational maintenance. The partitioning strategy divides data along two dimensions: time (for the transaction journal, which is append-heavy and queried by date range) and identity (for the balance table, which is update-heavy and queried by account ID). This dual strategy optimizes for the dominant access patterns of each table while enabling efficient data lifecycle operations.

7.1 Monthly Range Partitioning

The core_transactions table uses declarative monthly range partitioning on the transaction_timestamp column. Each monthly partition contains approximately 250 million rows (at current volume) and occupies approximately 4.7 TB of storage including indexes. The partitioning key choice (transaction_timestamp) aligns with the dominant query patterns: 94% of all queries on this table include a date range filter, enabling PostgreSQL's partition pruning to eliminate non-matching partitions from query execution plans entirely.

The partition scheme maintains 36 monthly partitions online (covering 3 years of data) plus a future partition that is pre-created one month in advance. The oldest partition is detached and exported to S3 (as Parquet via the aws_s3 extension) on a monthly schedule before being dropped from the database. This rolling window approach bounds the database's storage footprint at approximately 170 TB for the transaction data while maintaining the full 7-year retention in S3 for regulatory compliance.

Partition pruning provides dramatic query performance improvements. A query for transactions in the current month touches a single 4.7 TB partition rather than scanning the full 170 TB dataset -- a 36x reduction in data examined. For the common case of querying a single account's transactions in the past 30 days, the combination of partition pruning (eliminating 35 of 36 partitions) and the account_id B-tree index within the partition results in sub-5ms execution time regardless of the table's total size.

Index maintenance is performed at the partition level rather than the table level, which significantly reduces the operational impact of index operations. Creating a new index on the full core_transactions table would require scanning 18 billion rows -- an operation that would take days and consume massive I/O. With partitioning, a new index is created on each partition independently, and the operation on the current (actively written) partition uses CREATE INDEX CONCURRENTLY to avoid blocking write operations. Historical partitions can be indexed with standard (non-concurrent) CREATE INDEX since they receive no writes.

7.2 Hash Partitioning (Balances)

The account_balances table uses hash partitioning on the account_id column with 64 partitions. This table stores the current balance state for 94 million accounts and receives approximately 28,000 updates per second (one update per committed transaction to reflect the new balance). Hash partitioning was chosen because the access pattern is exclusively point lookups by account_id (there are no range scans on this table), and hash partitioning distributes the write load evenly across partitions to avoid hot spots.

The 64-partition count was determined through benchmarking to provide the optimal balance between write distribution (more partitions = less contention per partition) and query planning overhead (more partitions = more objects for the planner to consider). With 64 partitions, each partition contains approximately 1.47 million rows and occupies

approximately 800 MB of storage. The even distribution of accounts across partitions (verified by monitoring partition row counts) eliminates the hot-partition problem that would occur with range partitioning on account_id due to sequential account numbering.

The hash partitioning also enables parallel vacuum operations -- PostgreSQL's autovacuum workers can process multiple partitions simultaneously, which is critical for a table that generates approximately 2.4 billion dead tuples per day (28,000 updates/second * 86,400 seconds). Without partitioning, a single autovacuum process on a 94-million-row table would be unable to keep pace with dead tuple generation, leading to table bloat and performance degradation. With 64 partitions, up to 16 autovacuum workers operate concurrently (one per partition as needed), maintaining table bloat below 15%.

7.3 Partition Maintenance

Partition lifecycle operations are automated via pg_partman (the PostgreSQL partition management extension) combined with pg_cron for scheduling. The following maintenance operations run on a defined schedule without operator intervention:

- Pre-creation: New monthly partitions for core_transactions are created 30 days in advance, including all required indexes, constraints, and grants. This prevents any risk of insert failures due to a missing partition at month boundaries.

8 Data Lifecycle Management

The data lifecycle management strategy for the Transaction Database Cluster defines how transaction data moves through storage tiers over time, balancing the competing requirements of query performance (favoring hot, indexed storage), cost efficiency (favoring cheaper cold storage), and regulatory compliance (requiring 7-year retention for financial transaction records under Bank Secrecy Act and Regulation E).

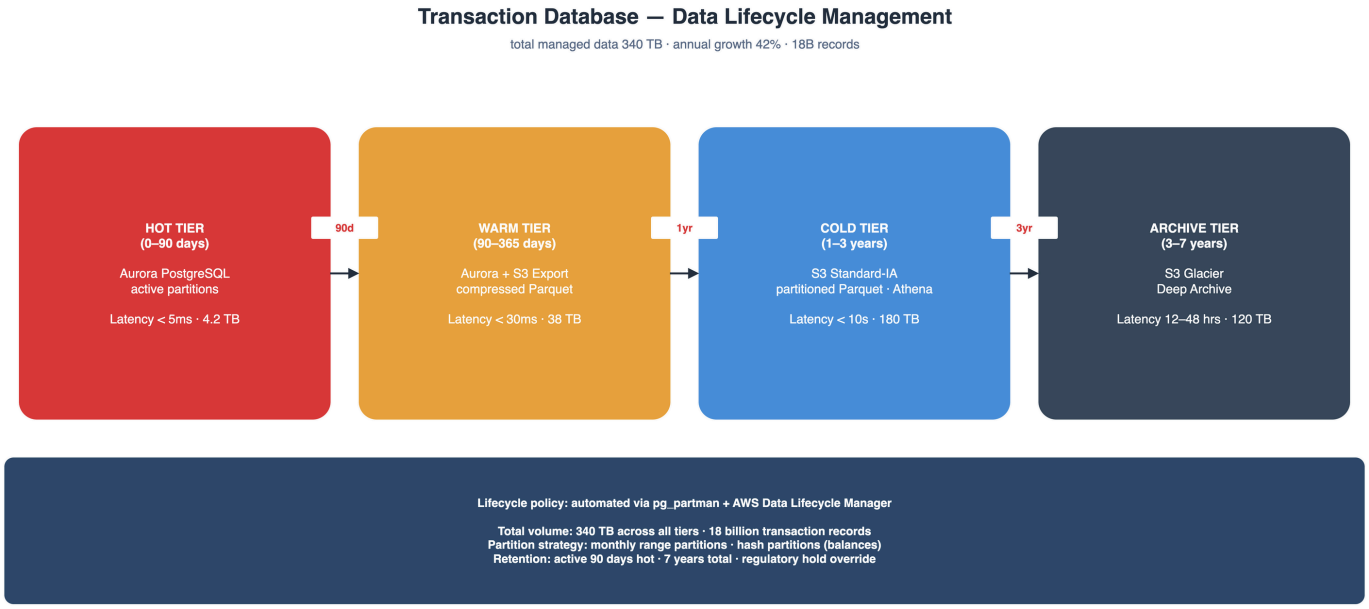


Figure 4: Data Lifecycle Management - Storage Tiers

The hot tier (0-90 days) consists of the most recent monthly partitions in Aurora PostgreSQL, totaling approximately 4.2 TB. These partitions are fully indexed with B-tree indexes on account_id, transaction_timestamp, and a composite covering index for the most common query patterns. The buffer pool hit ratio for hot data exceeds 98%, meaning nearly all queries are served from RAM without storage I/O. This tier serves all real-time transactional queries with sub-5ms latency.

The warm tier (90-365 days) represents partitions that remain in Aurora but have progressively fewer queries. As partitions age beyond 90 days, their indexes are reduced to only the account_id index (dropping the timestamp and covering indexes that are no longer needed because queries in this range are infrequent and can tolerate slightly higher latency). This index reduction saves approximately 30% of storage per partition. Queries against warm data typically complete in 20-50ms, which is acceptable for the statement generation and reporting use cases that access this age range.

The cold tier (1-3 years) represents data that has been exported from Aurora to S3 in Parquet format but remains queryable via Amazon Athena and Redshift Spectrum. The Parquet files are partitioned by year/month and compressed with Snappy, achieving an 8:1 compression ratio relative to the PostgreSQL on-disk format. The total cold tier volume is approximately 180 TB (compressed), stored in S3 Standard-Infrequent Access class. Queries against cold data complete in 5-30 seconds depending on the scan scope, which is acceptable for the regulatory investigation and audit use cases that occasionally access historical transactions.

The archive tier (3-7 years) represents the oldest retained transaction data, stored in S3 Glacier Deep Archive for minimum cost. This data is accessed only in extraordinary circumstances -- regulatory examinations, legal discovery, or fraud investigations that require historical records beyond the 3-year online window. Retrieval from Glacier Deep Archive requires 12-48 hours of advance notice, and the associated workflow includes automated compliance logging of who requested access and for what regulatory purpose. After the 7-year regulatory retention period expires, data is permanently deleted via S3 lifecycle rules with deletion confirmation logging.

9 Network Architecture

9.1 VPC Configuration

The Transaction Database Cluster resides in a dedicated VPC (10.200.0.0/16) that is separate from the application VPC to enforce network-level isolation between the database tier and the application tier. This separation ensures that the database's Security Groups cannot be inadvertently modified by application-tier automation and that network ACLs can enforce additional controls at the subnet boundary.

The database VPC is organized into three subnet tiers across three Availability Zones (9 subnets total). The data subnets (10.200.1.0/24 through 10.200.3.0/24) host the Aurora instances -- one subnet per AZ, as required by Aurora's multi-AZ deployment model. The proxy subnets (10.200.4.0/24 through 10.200.6.0/24) host the PgBouncer and ProxySQL instances, providing a dedicated network segment for the connection management layer. The management subnets (10.200.7.0/24 through 10.200.9.0/24) host monitoring agents, bastion hosts, and the Debezium CDC connector instances.

None of the subnets have internet gateway routes -- all outbound internet access (required for AWS API calls from PgBouncer IAM auth and Debezium metric publishing) is routed through NAT Gateways in the application VPC via the VPC peering connection. This design ensures that the database instances have no path to or from the public internet, even if a Security Group rule were misconfigured.

9.2 Security Groups

Security Groups enforce strict least-privilege network access at the instance level. The Aurora instances' Security Group allows inbound connections only on port 5432 from two source Security Groups: the PgBouncer Security Group (for pooled application traffic) and the Debezium Security Group (for CDC replication connections). No other source is permitted to connect directly to the database instances.

The PgBouncer Security Group allows inbound connections on port 6432 (PgBouncer's listening port) from the application VPC's CIDR range (10.100.0.0/16) via the VPC peering connection. This allows any application service in the application VPC to connect to PgBouncer, with application-level authentication (IAM tokens) providing the authorization layer. The ProxySQL Security Group is similar but listens on port 6033 and accepts connections only from the PgBouncer Security Group.

All Security Group rules are managed via Terraform with a strict change review process that requires approval from both the Database Reliability Engineering team and the Network Security team. Security Group changes are deployed during change windows and are automatically audited by AWS Config rules that alert on any non-compliant modification (such as adding a 0.0.0.0/0 inbound rule).

9.3 Cross-VPC Peering

Connectivity between the application VPC (10.100.0.0/16) and the database VPC (10.200.0.0/16) is provided by a VPC peering connection. The peering connection's route tables are configured with specific CIDR entries rather than full VPC CIDR routes, limiting cross-VPC communication to only the required subnet pairs. Specifically, only the application subnets (10.100.1.0/24 through 10.100.6.0/24) can route to the proxy subnets (10.200.4.0/24 through 10.200.6.0/24) in the database VPC. The data subnets hosting Aurora instances are not directly reachable from the application VPC.

The Direct Connect connection from DC-East terminates in the database VPC via a Virtual Private Gateway, providing the on-premises logical replica with direct network access to the Aurora writer's replication endpoint. The VPG route is specifically scoped to allow only the on-premises replica server's IP address (10.2.35.42/32) to connect on port 5432, preventing any other on-premises system from accessing the database directly. The Direct Connect link provides 2.1ms latency with 10 Gbps bandwidth, of which the logical replication consumes approximately 180 Mbps sustained.

10 Data Flow

10.1 Write Path

The write path begins when an application service (e.g., Payment Gateway) issues a transaction to insert or update records in the database. The request flows through the following stages: Application -> NLB -> PgBouncer (transaction mode, acquires server connection from pool) -> ProxySQL (identifies as write, routes to cluster endpoint) -> Aurora Writer Instance (executes the SQL statement and generates WAL records) -> Aurora Storage (replicates to 4 of 6 copies for write quorum) -> Commit acknowledgment returned to application.

The end-to-end write latency from application request to commit acknowledgment is typically 3-8ms for single-row INSERT operations and 5-15ms for multi-row transactions. The breakdown is approximately: NLB routing (0.1ms), PgBouncer connection acquisition (0.1-0.5ms including queue time), ProxySQL routing (0.1ms), Aurora query execution (1-3ms for simple INSERTs), and storage quorum acknowledgment (1-4ms depending on cross-AZ latency). For batch operations involving hundreds of rows within a single transaction, the execution phase dominates and total latency may reach 50-200ms.

Write operations are processed in strict serialization order at the writer instance using PostgreSQL's Multi-Version Concurrency Control (MVCC) with Serializable Snapshot Isolation (SSI) for critical operations (balance updates) and Read Committed isolation for append-only operations (transaction journal inserts). The SSI level is used specifically for the account_balances table to prevent lost-update anomalies where two concurrent transactions might both read the same balance and then both write a new balance without accounting for the other's update.

10.2 Read Path

The read path diverges based on the consistency requirement of the query. For strongly consistent reads (immediately after a write, such as balance verification), the query is routed to the writer instance via a connection-level flag that PgBouncer forwards to ProxySQL. For eventually consistent reads (the vast majority of queries), the path flows through: Application -> NLB -> PgBouncer -> ProxySQL (identifies as read, routes to reader endpoint) -> Aurora Reader Instance (one of three, selected by DNS round-robin at the reader endpoint level) -> response returned to application.

The end-to-end read latency for eventually consistent queries is typically 2-5ms for indexed point lookups and 10-50ms for range scans. The reader instances benefit from being dedicated to read workloads -- their buffer pools are not polluted by write transaction overhead (WAL buffers, transaction log, undo records), resulting in higher effective cache hit ratios than the writer instance. Reader-3 (analytics) may serve queries with latencies of seconds to minutes for complex aggregations, but these are isolated from OLTP traffic by the custom endpoint routing.

The replication lag between the writer and readers is monitored continuously via the aurora_replica_status system table. Under normal conditions, the lag is 10-20ms (representing the time for storage-layer replication and buffer cache invalidation). During write bursts (>25,000 writes/second), the lag may temporarily increase to 50-100ms before the readers catch up. Applications that cannot tolerate this lag window must use the strongly-consistent read path via the writer endpoint.

10.3 CDC Pipeline

The Change Data Capture (CDC) pipeline provides real-time streaming of database changes to downstream consumers

without requiring those consumers to query the database directly. This is architecturally critical because without CDC, the 23 downstream consumers would each need to poll the database for changes, creating an unsustainable read amplification that would require dozens of additional reader instances.

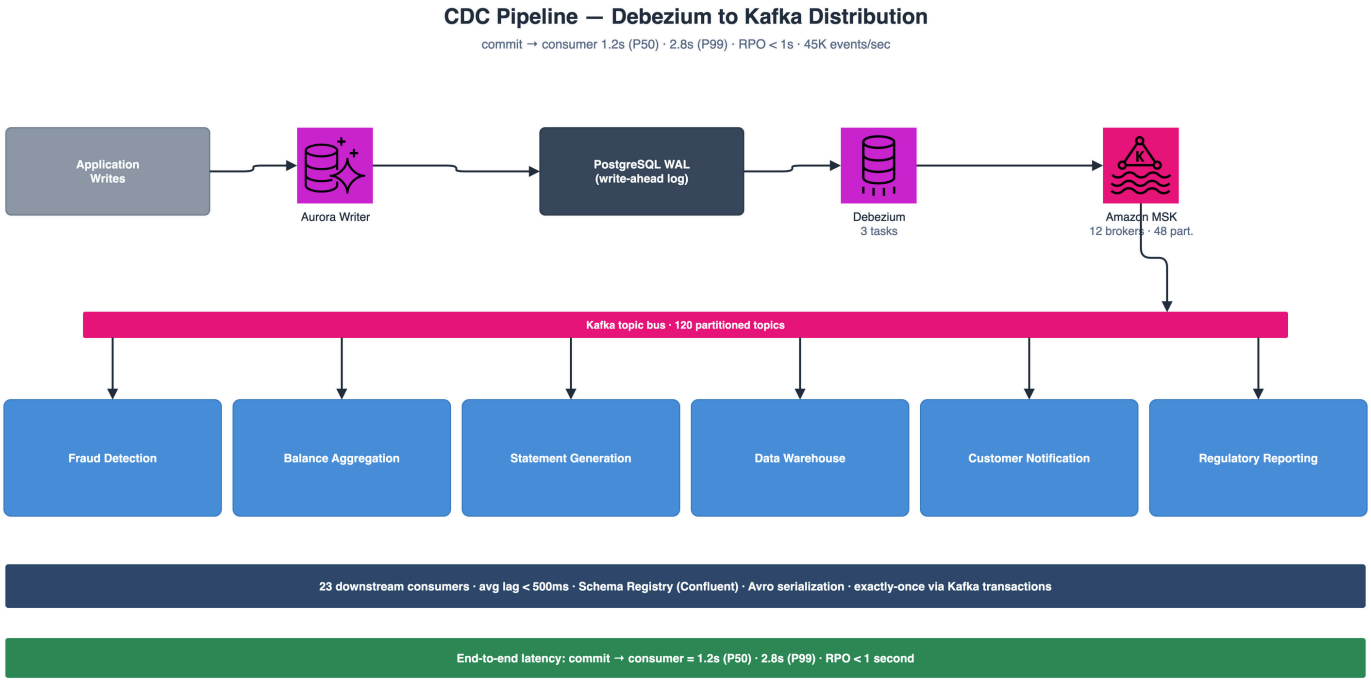


Figure 5: CDC Pipeline - Debezium to Kafka Distribution

The CDC pipeline uses Debezium, an open-source distributed platform for change data capture, deployed as a Kafka Connect connector running on 3 dedicated ECS tasks. Debezium connects to the Aurora writer instance using PostgreSQL's logical replication protocol (the pgoutput plugin) and captures all changes to the monitored tables as they are committed. Each change event includes the full after-image of the modified row, the operation type (create/update/delete), the transaction ID, the commit timestamp, and the schema version.

Captured events are published to Amazon MSK (Managed Streaming for Apache Kafka) with a 12-broker cluster providing 48 partitions across the transaction topics. The partition key is the account_id, ensuring that all changes for a given account are delivered in order to the same partition -- a requirement for consumers that maintain derived state (such as the balance aggregation service). The Kafka cluster retains events for 7 days, allowing consumers to replay events for recovery or backfill scenarios.

The end-to-end latency from a write committing on the Aurora writer to the event being available for consumption on Kafka is typically 1.2 seconds (P50) and 2.8 seconds (P99). The breakdown is: logical decoding and Debezium processing (400-800ms), Kafka producer batching and network (200-400ms), and broker replication acknowledgment (100-200ms). This latency meets the requirements of all 23 downstream consumers, whose SLAs range from 5 seconds (fraud detection) to 5 minutes (regulatory reporting).

11 Backup & Point-in-Time Recovery

The backup strategy for the Transaction Database Cluster leverages Aurora's continuous backup capability, which automatically backs up the entire storage volume to Amazon S3 incrementally and continuously -- there is no backup window, no performance impact, and no gap in backup coverage. This is fundamentally different from traditional PostgreSQL backup approaches (pg_dump, pg_basebackup, WAL archiving) that require explicit scheduling and consume database resources during execution.

Aurora's continuous backup enables point-in-time recovery (PITR) to any second within the configured retention period of 35 days. The PITR process creates a new Aurora cluster from the backup data at the specified point in time, leaving the original cluster unaffected. For the Transaction Database Cluster's 340 TB storage volume, PITR restoration takes approximately 15-25 minutes regardless of the target time (Aurora restores a consistent snapshot and then replays WAL records to reach the exact target timestamp).

In addition to continuous backup, manual snapshots are taken on a defined schedule to provide longer-term retention points that are not subject to the 35-day automatic retention window. Monthly snapshots are retained for 12 months, quarterly snapshots for 3 years, and annual snapshots for 7 years. These snapshots serve as recovery points for scenarios that are not addressable by PITR -- for example, recovering from a logical corruption introduced more than 35 days ago that was not immediately detected.

Backup Type	Frequency	Retention	RTO	RPO
Continuous (Aurora)	Continuous	35 days	15-25 min	< 5 seconds
Manual Snapshot	Monthly	12 months	15-25 min	Point-in-time
Quarterly Snapshot	Quarterly	3 years	15-25 min	Point-in-time
Annual Snapshot	Annually	7 years	15-25 min	Point-in-time
S3 Export (cold/archive)	Monthly	7 years	Hours (Athena)	Monthly boundary
Global DB (DR)	Continuous	Active	< 60 seconds	< 1 second

The PITR procedure is documented in a detailed runbook and tested quarterly. The most recent test (April 2026) successfully restored the cluster to a state 72 hours in the past, validated the restored data against checksums computed at the original time, and completed in 18 minutes. The test also validated that the restored cluster could serve production traffic by routing a percentage of read queries to it and comparing results with the production cluster.

For table-level recovery (restoring a single table rather than the entire cluster), the procedure involves PITR to a temporary cluster, exporting the target table via pg_dump, and importing it into the production cluster using pg_restore. This approach avoids the need to restore the entire cluster when only a single table is affected by corruption or accidental modification. Table-level recovery has been exercised twice in production (both due to erroneous batch jobs that modified data incorrectly) with recovery times of 35 and 42 minutes respectively.

12 Security & Access Control

Security for the Transaction Database Cluster is implemented in defense-in-depth layers spanning network isolation, authentication, authorization, encryption, and audit logging. The design addresses both external threats (unauthorized access from outside the bank's network) and internal threats (privilege escalation by authorized but malicious insiders), in compliance with OCC Heightened Standards, PCI-DSS v4.0, and SOX Section 404.

Network-level isolation is the first defense layer: the database instances reside in private subnets with no internet connectivity, accessible only from the proxy tier (PgBouncer/ProxySQL) and the CDC tier (Debezium), both of which are themselves in private subnets. This ensures that even if an application is compromised, the attacker cannot establish a direct connection to the database -- they must traverse the proxy layer, which provides an additional authentication checkpoint and query logging.

Authentication uses AWS IAM Database Authentication exclusively for application connections, as described in Section 6.3. For human administrative access (DBA operations), connections are permitted only through a bastion host in the management subnet, using SSH key authentication with MFA enforcement. Administrative database connections use a dedicated DBA role with time-limited session tokens generated via AWS Secrets Manager rotation, ensuring that long-lived DBA credentials do not exist.

Encryption at rest is provided by Aurora's native integration with AWS KMS using a customer-managed symmetric key (CMK). The CMK is configured with a 365-day automatic rotation policy, and key access is restricted to the Aurora service role and the Database Reliability Engineering team's administrative role. All 340 TB of storage data, automated backups, manual snapshots, and cross-region replication traffic are encrypted with this key.

Encryption in transit uses TLS 1.3 for all connections between applications and PgBouncer, between PgBouncer and Aurora, and between Aurora and the on-premises replica via Direct Connect. The Aurora cluster is configured with rds.force_ssl=1, which rejects any connection attempt that does not use TLS. Certificate verification is enforced at the client side using Amazon's RDS CA bundle, preventing man-in-the-middle attacks on the connection path.

Audit logging is implemented via the pgAudit extension, which records all DDL statements, all privileged operations (GRANT, REVOKE, CREATE ROLE), and sampled DML operations (1% random sampling to balance audit completeness with storage costs). Audit logs are shipped to CloudWatch Logs and then to the central SIEM (Splunk) with a retention of 7 years. The pg_stat_statements extension additionally tracks all distinct query fingerprints with execution statistics, enabling detection of anomalous query patterns that might indicate SQL injection or unauthorized data access.

13 Performance Tuning

Performance tuning for the Transaction Database Cluster is an ongoing discipline that encompasses PostgreSQL parameter optimization, indexing strategy, query analysis, and workload benchmarking. The performance targets are: P99 write latency below 10ms for single-row inserts, P99 read latency below 8ms for indexed point lookups, and sustained throughput of 28,000 writes/second and 95,000 reads/second without queuing.

The PostgreSQL parameter group is tuned for the specific hardware profile of Aurora's Graviton2 instances and the transaction workload characteristics. Key parameters and their rationale are documented below:

Parameter	Value	Rationale
shared_buffers	128 GB	25% of 512 GB RAM; Aurora storage is fast but buffer hits avoid network I/O
effective_cache_size	384 GB	Guides planner cost estimates; includes OS page cache
work_mem	256 MB	Supports in-memory sorts for complex queries; monitored for temp file spills
maintenance_work_mem	2 GB	Accelerates VACUUM, CREATE INDEX; bounded to prevent OOM
random_page_cost	1.1	Aurora storage latency is uniform; near-sequential cost guides index usage
effective_io_concurrency	200	Aurora supports massive parallel I/O; maximizes prefetch
max_parallel_workers_per_gather	4	Enables parallel seq scans for analytics; bounded for OLTP
wal_buffers	64 MB	Reduces WAL write contention at high write rates
checkpoint_timeout	15 min	Spreads checkpoint I/O; Aurora handles durability via storage layer
autovacuum_max_workers	16	Matches partition count for parallel vacuum across hash partitions

Indexing strategy follows a principle of minimal indexes with maximum coverage. Each index consumes storage, increases write latency (due to index maintenance), and adds vacuum overhead. Therefore, indexes are added only when supported by query plan analysis showing that the index eliminates a sequential scan or provides an order-of-magnitude improvement in row filtering.

The primary indexes on the core_transactions table are: (1) a B-tree on (account_id, transaction_timestamp DESC) that covers the dominant query pattern of 'most recent transactions for an account'; (2) a B-tree on (transaction_id) for unique lookups by transaction identifier; and (3) a partial index on (transaction_status) WHERE transaction_status = 'pending' that optimizes the payment settlement workflow's query for unprocessed transactions. The partial index covers only 0.02% of rows (approximately 3.6 million pending transactions at any time) rather than all 18 billion rows, providing sub-millisecond lookups with minimal storage overhead.

Query optimization reviews are conducted weekly by the Database Reliability Engineering team using pg_stat_statements data. The top 10 queries by total execution time are reviewed for optimization opportunities -- plan analysis, missing indexes, unnecessary sorts, and opportunities to push filters earlier in the execution plan. Over the past quarter, this process has identified and resolved 14 query performance issues, collectively reducing average query response time by 23% without any schema changes or hardware upgrades.

14 Monitoring & Alerting

The monitoring architecture provides comprehensive visibility into the health and performance of all Transaction Database Cluster components: Aurora instances (writer and readers), PgBouncer connection pools, ProxySQL query routing, the Debezium CDC pipeline, and the on-premises logical replica. Monitoring data is collected by Amazon CloudWatch and a Prometheus/Grafana stack, with alerts routed through PagerDuty for on-call notification and escalation.

Database-level metrics are collected at 1-second granularity via Enhanced Monitoring (OS-level metrics including CPU, memory, disk I/O, network, and process counts) and at 1-minute granularity via CloudWatch (Aurora-specific metrics including buffer cache hit ratio, active sessions, deadlocks, and replication lag). Custom metrics are published by a Prometheus exporter running in the management subnet that queries pg_stat_activity, pg_stat_statements, and the Aurora system tables every 10 seconds.

Metric	Warning Threshold	Critical Threshold	Current Value
Writer CPU Utilization	> 70%	> 85%	41%
Writer Active Sessions	> 150	> 300	78
Buffer Pool Hit Ratio	< 97%	< 95%	98.7%
Replication Lag (readers)	> 50ms	> 200ms	14ms
Replication Lag (DR)	> 2 seconds	> 5 seconds	620ms
PgBouncer Pool Saturation	> 75%	> 90%	34%
CDC Pipeline Lag	> 5 seconds	> 30 seconds	1.2s
Deadlocks/minute	> 5	> 20	0.3
Connection Count (total)	> 3500	> 4500	1,840
Transaction Rollback Rate	> 2%	> 5%	0.4%
Disk Queue Depth	> 50	> 100	12
On-Prem Replica Lag	> 30 seconds	> 60 seconds	4.2s

PgBouncer metrics are collected via the SHOW STATS and SHOW POOLS commands, exposed to Prometheus via pgbouncer_exporter. Key metrics include: client connections waiting (indicates pool exhaustion), average query duration (indicates database slowness), and server connections in login state (indicates authentication issues or database unavailability). A dedicated Grafana dashboard displays the connection flow from clients through PgBouncer to Aurora, enabling rapid diagnosis of connection-layer issues.

CDC pipeline monitoring uses Kafka consumer group lag as the primary health indicator. The Debezium connector publishes its own metrics (rows captured per second, snapshot progress, replication slot WAL size) to a dedicated Prometheus endpoint. If the WAL size consumed by the replication slot exceeds 100 GB (indicating Debezium is falling behind and WAL is accumulating on the writer), a critical alert fires because unchecked WAL growth could eventually fill the storage volume and halt all write operations.

15 Operational Procedures

This section documents the recurring operational procedures for the Transaction Database Cluster, including maintenance schedules, emergency procedures, and patching workflows. All procedures are maintained as runbooks in the internal operations wiki and are reviewed quarterly for accuracy.

15.1 Maintenance Schedule

Routine maintenance is performed during the defined maintenance window: Sundays 02:00-06:00 UTC, which corresponds to Saturday 9:00 PM - 1:00 AM ET -- the lowest traffic period with sustained write volume below 2,000 operations per second. The following operations are scheduled within this window:

- Aurora engine minor version patches (applied by AWS, typically every 4-6 weeks). Patches are applied to readers first, then the writer with a brief failover (< 30s).

15.2 Emergency Procedures

The following emergency procedures are documented in the on-call runbook and have been validated through quarterly game day exercises:

Writer Instance Failure: Automatic failover to Reader-1 within 30-60 seconds. On-call verifies via CloudWatch that the new writer is accepting connections and that PgBouncer has reconnected. Post-failover action: verify CDC pipeline has reconnected, verify on-premises logical replication has resumed, monitor for any transaction failures during the failover window that need replay.

PgBouncer Fleet Degradation: If multiple PgBouncer instances become unhealthy simultaneously, applications experience connection timeouts. Emergency action: verify NLB health check status, restart unhealthy instances via ASG instance refresh, and if necessary, temporarily allow direct application connections to Aurora (bypassing PgBouncer) using the emergency bypass Security Group rule. The bypass rule is pre-defined but disabled; enabling it requires approval from the incident commander.

CDC Pipeline Stall: If Debezium stops capturing changes (evidenced by consumer lag increasing continuously), the emergency procedure is: (1) Check Debezium connector status via the Kafka Connect REST API. (2) If the connector has failed, attempt restart. (3) If restart fails, check the replication slot status on Aurora -- if the slot has been invalidated (WAL recycled), a snapshot re-sync is required. (4) If WAL accumulation exceeds 100 GB, assess whether to drop the replication slot (losing CDC events) to protect the writer from storage exhaustion.

Cross-Region Failover Decision: When the primary region (us-east-1) is confirmed unavailable for > 5 minutes, the incident commander and on-call DBA jointly decide whether to invoke cross-region failover. The decision considers: (1) Is AWS reporting a regional incident? (2) What is the estimated time to recovery? (3) Is the RPO (< 1 second of data loss) acceptable for the current transaction volume? After approval, execute the failover

runbook (estimated time: 60 seconds for promotion, 5 minutes for full traffic migration including DNS propagation).

15.3 Patching Strategy

Aurora PostgreSQL engine patches are applied on a monthly cadence during the maintenance window. The patching process follows a rolling upgrade pattern: (1) Apply patch to Reader-3 (analytics) first and monitor for 30 minutes. (2) Apply to Reader-1 and Reader-2 sequentially with 15-minute observation between each. (3) Apply to the writer last, which triggers a brief failover to an already-patched reader. This rolling approach ensures that at least one instance is running the known-good version at all times during the patching process.

Major version upgrades (e.g., PostgreSQL 15 to 16) follow a blue-green deployment pattern: a new cluster is created with the target version, data is replicated via logical replication from the current cluster, and traffic is switched after validation. This approach provides instant rollback capability (simply switch traffic back to the original cluster) and avoids the multi-hour in-place upgrade downtime that would be required for a 340 TB storage volume. The most recent major version upgrade (PostgreSQL 14 to 15, November 2025) completed with zero downtime and zero data loss using this blue-green approach.

16 Known Risks & Deviations

The following risks and architectural deviations have been identified, assessed, and formally accepted through the Architecture Review Board (ARB). Each item includes the risk description, current mitigation strategy, and responsible owner.

R-001: Single-writer bottleneck: Aurora PostgreSQL supports only a single writer instance per cluster, creating a theoretical ceiling on write throughput. Current peak utilization is 28,000 writes/second against a tested maximum of 45,000 writes/second, providing 60% headroom. However, at the current 42% annual growth rate, the ceiling will be reached by approximately Q2 2028. Mitigation: The team is evaluating Aurora Limitless Database (multi-writer sharding, currently in preview) and application-level sharding as potential solutions. A decision on the scaling approach will be made by Q4 2026 to allow implementation time before the ceiling is reached. Status: Monitoring. Owner: Database Platform Engineering.

R-002: Logical replication slot WAL accumulation: The logical replication slot for the on-premises replica prevents WAL recycling on the writer instance. If the replica becomes unavailable for an extended period, WAL accumulates on the writer and could eventually exhaust storage. Mitigation: A monitoring alert fires at 50 GB of accumulated WAL (critical at 100 GB). The automated runbook drops the slot if accumulation reaches 100 GB, accepting the loss of CDC events that will require a full re-sync. A 24-hour WAL retention policy provides a grace period for transient outages. Status: Accepted (ARB-2025-142). Owner: Database Reliability Engineering.

R-003: PgBouncer transaction mode limitations: Transaction mode pooling prevents use of prepared statements, advisory locks, and session-level SET commands. While all current applications are designed for this constraint, new application integrations occasionally require education about these limitations. Mitigation: Integration checklist includes PgBouncer compatibility validation. Applications requiring session-level features can connect via a dedicated PgBouncer pool configured in session mode (limited to 100 connections). Status: Accepted (ARB-2025-078). Owner: Database Platform Engineering.

R-004: Cross-region failover data loss window: Cross-region failover to us-west-2 incurs data loss equal to the replication lag at the time of failure (typically 400-800ms of committed transactions). For a peak write rate of 28,000 transactions/second, this represents 11,000-22,000 potentially lost transactions. Mitigation: (1) The CDC pipeline to Kafka provides an independent record of all committed transactions that can be replayed after failover. (2) Application-level idempotency keys enable safe replay without creating duplicates. (3) The reconciliation process between the CDC stream and the database detects and repairs any discrepancies within 15 minutes of failover completion. Status: Accepted (ARB-2025-156). Owner: Cloud Architecture.

R-005: On-premises replica decommissioning dependency: The on-premises PostgreSQL replica exists solely for 5 legacy applications scheduled for decommissioning. If the decommissioning timeline slips (as it has twice already), the replica infrastructure must be maintained indefinitely. The replica hardware is approaching end-of-life (warranty expires Q1 2027) and will require replacement if legacy application migration is not completed by then. Mitigation: Executive escalation of decommissioning timeline. Hardware replacement RFP prepared as contingency. Status: At Risk. Owner: Application Modernization (with escalation to CTO office).

R-006: Partition explosion risk for core_transactions: At 36 monthly partitions with indexes, the core_transactions table has over 250 relational objects that PostgreSQL's planner must consider. Beyond 100+ partitions, planning time increases non-linearly. Current planning time for typical queries is 2-3ms, which is acceptable, but extending the retention window or switching to daily partitions would significantly increase

planning overhead. Mitigation: Maintain monthly partitioning granularity and 36-partition rolling window. If finer granularity is needed, evaluate native partitioning alternatives or partition-level routing at the application layer. Status: Monitoring. Owner: Database Platform Engineering.

References

- Amazon Aurora PostgreSQL Developer Guide (AWS Documentation)